

Cold Chain Control Tower

Business Presentation

Ujjwal Khanna · September 2026

An autonomous AI copilot that keeps refrigerated freight compliant — and lets anyone interrogate fleet data in plain English.

Agenda

- 1. The Business Problem** What cold-chain failures actually cost, and why they are caught late.
- 2. The Operational Bottleneck** How a dispatcher handles a temperature drift today, step by painful step.
- 3. The Solution** A conversational compliance agent that chains telemetry, weather and SOP retrieval.
- 4. Governance & Security** Least-privilege database access, a semantic isolation layer, and a complete audit trail.
- 5. What It Does Not Do** The boundaries of the current system, stated plainly.

1. The Business Problem

The high cost of spoilage

Perishable cargo has a narrow tolerance. For fresh perishables the governing SOP allows a band of **0°C to 4°C**. Above 4°C a cold-chain breach is declared and the load is at risk.

Three things make this expensive:

- **Detection is late.** A drift is noticed when someone happens to look at a dashboard.
- **Judgement is slow.** Deciding what to do means cross-referencing telemetry, external conditions, and a policy document that is revised frequently.
- **The rules move.** SOPs are updated faster than the people applying them can re-learn them, so decisions drift out of compliance without anyone noticing.

The result is spoiled cargo, SLA penalties, and inconsistent escalation.

2. The Operational Bottleneck

The cost of manual friction

What a dispatcher does today when a reefer unit starts drifting:

1. **Detect** — Notice a temperature drift, usually from a dashboard or a driver call.
2. **Query** — Find the affected shipments. This means either knowing SQL or waiting on someone who does.
3. **Correlate** — Check external conditions. Is this a heat event, or has the cooling unit failed? The answer changes the entire response.
4. **Search** — Locate the governing SOP clause and confirm the current threshold and mitigation.
5. **Escalate** — Determine the correct tier and execute.

Every step is a context switch between a different system. The delay between step 1 and step 5 is where cargo is lost.

3. The Solution

A conversational compliance agent

One question, in plain English, replaces the whole sequence:

"Find any active shipments near Los Angeles. Check the local weather there, and tell me if the current cargo temperature violates the SOP for fresh perishables."

The agent decides for itself that this requires three different systems, calls them in dependency order, and returns a decision — not a dashboard.

Three capabilities, chained autonomously:

Capability	What it does
Text-to-SQL telemetry	Translates the question into T-SQL and runs it against a read-only view. No SQL knowledge required of the user.
Live corridor conditions	Retrieves real-time weather and wind for any coordinates and derives a transit disruption index.
Compliance retrieval	Pulls the governing SOP clause from a vector index so every recommendation is cited.

Every answer has the same shape

- 1 **Executive Summary** — the anomaly, and the immediate operational risk
- 2 **Telemetry & Environment Analysis** — the readings, and whether the cause is environmental or mechanical
- 3 **Required Action Plan** — numbered steps, each traced to the SOP rule behind it

This is the difference between an answer and a decision. In testing, the agent correctly distinguished a **refrigeration failure** from a heat event by observing that cargo temperature was far above ambient — the diagnosis that determines whether you call the driver or reroute the load.

Knowing when *not* to act

Asked a policy question that needs no data — *"what is the difference between a Tier 1 and Tier 2 escalation?"* — the agent performs a single retrieval and answers. Restraint matters as much as capability: an agent that queries everything on every request is expensive and slow.

4. Governance & Security

Built so the AI cannot do damage

1. Least-privilege role The agent connects as a dedicated account with `SELECT` on exactly one view and nothing else. The raw table is explicitly denied, as is every write to the wider schema.

2. A semantic isolation layer The agent never sees the real database. It queries a view that renames legacy columns into readable English — which both improves reasoning quality and means the agent has no name by which to reference the underlying tables.

3. Enforcement at the database, not the prompt This is the critical distinction. A hallucinated `DROP TABLE` is rejected by the database engine. Guardrails that live only in a prompt can be talked around; grants cannot.

4. A complete audit trail Every reasoning step, tool call, argument set, raw result and final answer is written to a SQL audit table. The agent holds `INSERT` rights only, so it cannot alter or erase its own history. Any answer can be replayed and explained after the fact.

5. What It Does Not Do

Stating the boundaries honestly, because an overclaimed capability is a liability in production.

- **No human-in-the-loop gate.** The agent is read-only and advisory. It *recommends* an escalation; a person still performs it. There is no approval step in the workflow.
- **No autonomous action.** It cannot dispatch, reroute, or modify any record.
- **Append-only, not tamper-proof.** The audit trail cannot be altered by the agent. A database administrator retains full rights over it.
- **Demonstration data.** The underlying dataset is historical and its GPS coordinates are synthetically generated, so distances and routes are illustrative rather than physically meaningful. The architecture, security model and reasoning behaviour are real.

The takeaway

The hard part of this project was never the model.

It was giving a language model useful access to enterprise data **without giving it dangerous access** — a semantic layer that makes a legacy schema legible, a permission model enforced by the database rather than by instructions, and an audit trail that makes every automated decision reviewable.

That combination is what makes a system like this deployable rather than merely demonstrable.

Thank you.

Ujjwal Khanna · github.com/UjjwalK08/supply-chain-logisitics-FDE-project